UNDEF

FADENA
Facultad de Defensa Nacional

Facultad de Defensa Nacional

Carrera: Lic. en Ciberdefensa

Materia
Sistemas Operativos I

Unidad 4 / Semana 10

Año: 2025

Contenidos

Técnicas de administración de memoria. Espacio de direcciones. Intercambio. Direccionamiento físico y lógico. Particiones.

Índice

1. Administración de la Memoria 3
1.1 Direcciones de memoria físicas y lógicas 4
Dirección Física 4
Dirección Lógica (o Virtual) 4
¿Por qué esta distinción es importante? 5
1.2 Espacio de direccionamiento 5
1.3 Requisitos de la administración de la memoria 6
2. Sistemas Monoprogramados 10
3. Sistemas Multiprogramados 10
3.1 Esquemas de asignación de memoria 11
3.1.1 Particiones Fijas 11
3.1.1.1 Mismo Tamaño 12
3.1.1.2 Diferentes Tamaños 12
3.1.2 Particiones Variables 14
3.1.2.1 Algoritmos de reubicación en particiones dinámicas 16
3.1.2.2 Fragmentación y Compactación 18
3.1.3 Cuadro Comparativo 18

1. Administración de la Memoria

La memoria constituye un componente esencial en un sistema informático, cuya administración por parte del sistema operativo, a través del gestor de memoria, tiene como objetivo la asignación de memoria principal a los procesos que la requieran.

La gestión de memoria debe alcanzar los siguientes objetivos:

- **Ofrecer a cada proceso un espacio lógico propio:** Esta abstracción, gestionada por el sistema operativo, hace que cada proceso perciba que tiene acceso a toda la memoria, aunque en realidad esta se comparta físicamente con otros procesos. Esto simplifica mucho la programación.

- **Proporcionar protección entre procesos:** Es decisivo para la estabilidad y seguridad del sistema. El gestor de memoria asegura que un proceso no pueda leer o escribir en el espacio de memoria de otro proceso sin permiso, previniendo fallos o accesos no autorizados.

- **Permitir que los procesos compartan memoria:** Aunque la protección es importante, hay situaciones donde los procesos necesitan colaborar y compartir datos de forma eficiente. El gestor de memoria facilita la creación de regiones de memoria compartida, lo que es crucial para la comunicación interproceso.

- **Dar soporte a las distintas regiones del proceso:** Un proceso no es solo un bloque de código. Generalmente, está compuesto por diferentes secciones (código, datos, pila[1], etc.). El gestor de memoria es responsable de organizar y manejar estas distintas regiones de manera apropiada.

- **Maximizar el rendimiento del sistema:** Todos los objetivos anteriores contribuyen a este fin. Un buen gestor de memoria optimiza el uso de la memoria física, reduce la fragmentación, minimiza los accesos a disco (en el caso de memoria virtual) y asegura que los procesos se ejecuten de manera eficiente.

[1] Dentro de las diferentes secciones que componen un proceso en ejecución, la **pila** es una región de memoria dinámica que crece y decrece a medida que el programa realiza llamadas a funciones y declara variables locales. Es crucial para el seguimiento del flujo de ejecución de un programa.

- **Proporcionar a los procesos mapas de memoria muy grandes:** Esto se logra principalmente a través de la **memoria virtual**. Permite que los programas utilicen un espacio de direcciones mucho mayor que la memoria física disponible, creando la ilusión de una memoria casi ilimitada para el programador.

1.1 Direcciones de memoria físicas y lógicas

En el contexto de la gestión de memoria en los sistemas operativos, es fundamental comprender la diferencia entre **dirección lógica** (o virtual) y **dirección física**. Estas dos direcciones son clave para que los programas puedan funcionar de manera eficiente y segura en un sistema con memoria limitada y múltiples procesos.

Dirección Física

La **dirección física** es la dirección real y única de una ubicación en la memoria principal (RAM) de la computadora. Es la dirección que el controlador de memoria ve y utiliza para acceder a los datos.

- **Ubicación real:** Representa la posición exacta de los datos en los chips de memoria.
- **Vista por la unidad de memoria:** Esta dirección es la que se envía al hardware de la memoria para leer o escribir datos.
- **Mapeo:** La **Unidad de Gestión de Memoria (MMU)**, que es un componente de hardware del procesador, es la encargada de traducir las direcciones lógicas generadas por la CPU a direcciones físicas.
- **Espacio de direcciones físicas:** El conjunto de todas las direcciones físicas disponibles en el sistema se denomina **espacio de direcciones físicas**.

Dirección Lógica (o Virtual)

La **dirección lógica** es la dirección que genera la CPU (Unidad Central de Procesamiento) para un programa. También se le conoce como **dirección virtual** porque no corresponde directamente a una ubicación real en la memoria física.

- **Generada por la CPU:** Cuando un programa se ejecuta, la CPU trabaja con estas direcciones lógicas.
- **Independiente de la memoria física:** Un programa no "sabe" dónde se encuentra realmente en la RAM. Opera en un espacio de direcciones lógicas

que es independiente del espacio de direcciones físicas. Esto permite que el sistema operativo reubique un proceso en la memoria física sin que el programa necesite ser modificado.

- **Abstracción:** Proporciona una capa de abstracción entre el programa y el hardware de la memoria. Esto es crucial para la multiprogramación, ya que varios programas pueden usar el mismo rango de direcciones lógicas sin interferir entre sí.
- **Espacio de direcciones lógicas:** El conjunto de todas las direcciones lógicas generadas por un programa se denomina **espacio de direcciones lógicas**. Este espacio puede ser mucho más grande que la memoria física disponible.

¿Por qué esta distinción es importante?

La distinción entre direcciones lógicas y físicas es fundamental para la gestión de memoria en los sistemas operativos por varias razones:

1. **Protección:** Permite que cada proceso tenga su propio espacio de direcciones lógicas, lo que evita que un proceso acceda o modifique accidentalmente la memoria de otro proceso.
2. **Multiprogramación:** Facilita que múltiples procesos se ejecuten simultáneamente, incluso si la memoria física disponible es limitada. El sistema operativo puede cargar y descargar partes de los programas en memoria según sea necesario.
3. **Flexibilidad:** Permite que un programa se cargue en cualquier parte de la memoria física disponible, sin necesidad de que esté en una ubicación contigua o predefinida. La MMU se encarga de realizar la traducción en tiempo real.
4. **Memoria Virtual:** La implementación de direcciones lógicas y físicas es la base de la **memoria virtual**. Esto permite que un sistema operativo simule tener más memoria RAM de la que realmente posee, moviendo datos entre la RAM y el almacenamiento secundario (como el disco duro) de forma transparente para los programas.

1.2 Espacio de direccionamiento

El **espacio de direccionamiento** (o espacio de direcciones) es el rango de direcciones de memoria al que un procesador o un programa puede acceder.

Imagina que es como el mapa que un programa ve de la memoria. Este mapa puede ser de dos tipos principales:

1. **Espacio de Direccionamiento Lógico (o Virtual)**: Este es el "mapa" que cada programa individual percibe. Para un programa, parece que tiene acceso a un rango de direcciones de memoria continuo y exclusivo, desde 0 hasta un máximo. Sin embargo, este espacio es una abstracción. La mayoría de los sistemas operativos modernos utilizan memoria virtual, lo que permite que el tamaño total de los espacios de direccionamiento lógicos de todos los programas en ejecución sea mucho mayor que la cantidad de memoria física real disponible en la computadora. El gestor de memoria es el encargado de traducir estas direcciones lógicas a direcciones físicas reales.
2. **Espacio de Direccionamiento Físico**: Este es el "mapa" de la memoria RAM real y tangible instalada en la computadora. Las direcciones físicas corresponden directamente a las ubicaciones reales en los chips de memoria. Solo hay un espacio de direccionamiento físico.

La clave es que el gestor de memoria del sistema operativo se encarga de realizar la traducción entre el espacio de direccionamiento lógico de cada proceso y el espacio de direccionamiento físico de la máquina. Esto permite que múltiples programas se ejecuten simultáneamente sin interferir entre sí y que los programas puedan ser más grandes que la memoria física disponible.

En resumen, el espacio de direccionamiento es la vista organizada y accesible de la memoria para un procesador o programa, ya sea la vista abstracta y privada de un proceso (lógica) o la vista concreta de la memoria RAM (física).

1.3 Requisitos de la administración de la memoria

Los objetivos mencionados definen los requisitos que la gestión de memoria debe satisfacer. Vamos a clasificarlos y analizarlos.

Requisitos de la Gestión de Memoria:

a) Reubicación
b) Protección
c) Compartición

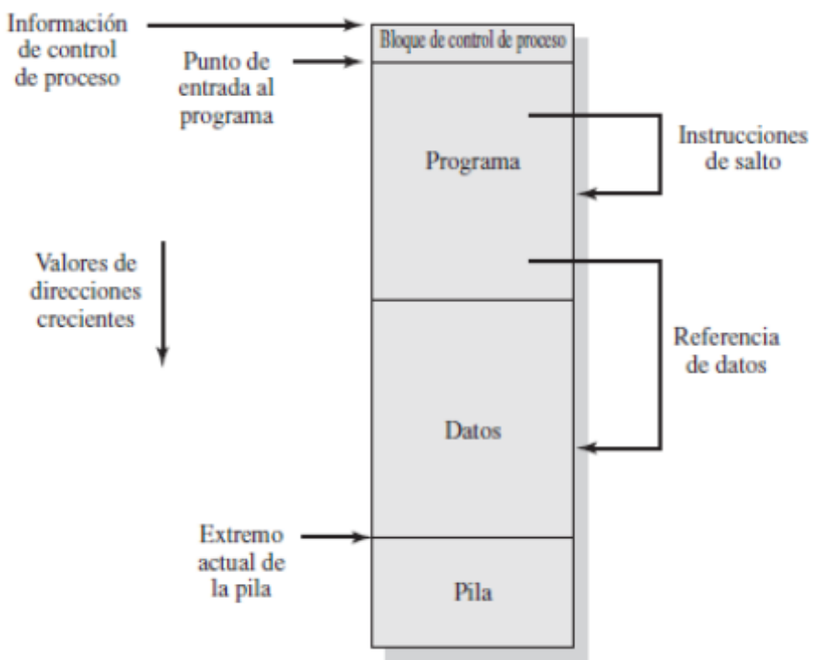
d) Organización lógica

e) Organización física

- **Reubicación**: Los programas no siempre pueden cargarse en la misma posición de memoria cada vez que se ejecutan. El gestor de memoria se encarga de que los programas puedan residir en diferentes partes de la memoria en distintos momentos y de ajustar las referencias a memoria dentro del código para que siempre apunten a la ubicación correcta. Esto es fundamental en sistemas con memoria virtual y para la multitarea.

Existen 3 situaciones posibles en relación a los programas y su reubicación:

  - ***Sin reubicación***, en este caso se usan direcciones absolutas de memoria y siempre el programa se cargará en la misma dirección de memoria y se define en tiempo de compilación. Ejemplo: .com en MS-DOS
  - ***Reubicación estática***, en este caso el compilador genera código relocalizable, que crea las direcciones absolutas de memoria en el momento de la carga del programa antes de la ejecución.
  - ***Reubicación dinámica***, en este caso la MMU traduce las direcciones lógicas a físicas en tiempo real permitiendo que el proceso pueda cambiar de ubicación en la memoria.



Requisitos de direccionamiento para un proceso[2]

[2] Stallings, William (2005), Sistemas operativos. Aspectos internos y principios de diseño, Quinta edición.

- **Protección**: Un proceso no debe poder acceder a la memoria asignada a otro proceso sin permiso, ni tampoco a la memoria del propio sistema operativo. El gestor de memoria implementa mecanismos de protección para evitar que código malicioso o erróneo de un programa interfiera con otros programas en ejecución o con la estabilidad del sistema.

- **Memoria Compartida:** Aunque la protección es importante, en ocasiones diferentes procesos necesitan compartir información y, para ello, deben poder acceder a la misma sección de memoria. El gestor de memoria facilita la creación y gestión de áreas de memoria compartida, lo que es una forma eficiente de comunicación entre procesos.

- **Organización lógica:** Los programas modernos no son bloques de código monolíticos, sino que están organizados lógicamente en **módulos** (como el código ejecutable, los datos y las bibliotecas). Estos módulos pueden tener diferentes propiedades: ser compartidos entre varios procesos, ser de solo lectura o ser modificables. El **gestor de memoria** es el responsable de manejar esta organización lógica, lo cual contrasta con la visión lineal y física de las direcciones de memoria.

Para lograr esta organización lógica, se emplean técnicas como la **segmentación de memoria**. Esta permite que los módulos sean escritos y compilados de forma independiente. Además, facilita que todas las referencias entre módulos se resuelvan dinámicamente en tiempo de ejecución, lo que añade flexibilidad al sistema.

- **Organización Física**: la memoria de una computadora se organiza en al menos dos niveles, conocidos como **memoria principal** y **memoria secundaria**. La memoria principal proporciona acceso rápido a un costo relativamente alto. Adicionalmente, la memoria principal es volátil; es decir, no proporciona almacenamiento permanente. La memoria secundaria es más lenta y más barata que la memoria principal y normalmente no es volátil. Por tanto, la memoria secundaria de larga capacidad puede proporcionar almacenamiento para programas y datos a largo plazo, mientras que una memoria principal más pequeña contiene programas y datos actualmente en uso.

En este esquema de dos niveles, la organización del flujo de información entre la memoria principal y secundaria supone una de las preocupaciones principales del sistema. La responsabilidad para este flujo podría asignarse a cada programador en particular, pero no es practicable o deseable por dos motivos:

1. La memoria principal disponible para un programa más sus datos podría ser insuficiente. En este caso, el programador debería utilizar una técnica conocida como **superposición** (*overlaying*), en la cual los programas y los datos se organizan de tal forma que se puede asignar la misma región de memoria a varios módulos, con un programa principal responsable para intercambiar los módulos entre disco y memoria según las necesidades. Incluso con la ayuda de herramientas de compilación, la programación con overlays malgasta el tiempo del programador.

2. En un entorno multiprogramado, el programador no conoce en tiempo de codificación cuánto espacio estará disponible o dónde se localizará dicho espacio.

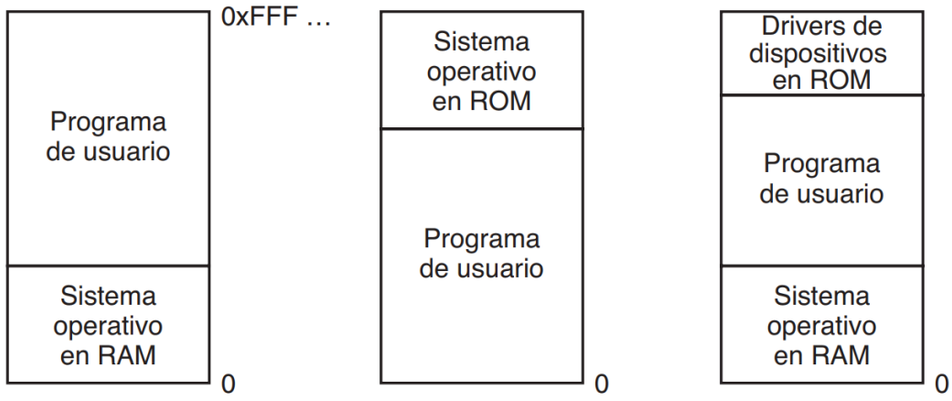
Por tanto, está claro que la tarea de mover la información entre los dos niveles de la memoria debería ser una responsabilidad del sistema. Esta tarea es la esencia de la gestión de la memoria.

2. Sistemas Monoprogramados

En los sistemas monoprogramados, la memoria se divide en dos partes:

- Una parte para el sistema operativo
- la otra parte es el programa en ejecución

Para optimizar el espacio de memoria, el SO podría residir en la parte baja de la memoria o en la parte alta y dejar el resto para uso de los programas a ejecutar. En ambos casos es necesario asegurar la protección de estas particiones, es decir que uno no acceda a las áreas de memoria del otro.



Tres formas simples de organizar la memoria con un SO y un proceso de usuario[3]

3. Sistemas Multiprogramados

En los sistemas multiprogramados se desea tener varios programas en memoria para que se puedan ejecutar en forma concurrente. Para eso se implementaron diversos esquemas de administración de memoria, algunos usan sólo la memoria principal y otros, en cambio, se complementan con el uso de la memoria secundaria como una opción cuando la memoria principal está totalmente ocupada.

Estos mecanismos son conocidos como swap o intercambio y memoria virtual.

El **intercambio** o **swap** es una técnica de administración de memoria que permite optimizar el uso de la **RAM** al trasladar temporalmente los datos de procesos inactivos o menos utilizados a un área específica del disco denominada **memoria de intercambio**. Esto libera espacio en la memoria principal y permite que el sistema continúe ejecutando nuevos procesos sin problemas.

Cuando la RAM alcanza su límite, el sistema operativo identifica las tareas menos activas y las transfiere al swap. Luego, si es necesario, recupera esos datos para reubicarlos en la RAM y continuar la ejecución del proceso.

El concepto de memoria virtual lo abordaremos más adelante.

[3] Tanenbaum Andrew S. (2009), **Sistemas Operativos Modernos**, tercera edición, Editorial Pearson Educación, México D.F.

3.1 Esquemas de asignación de memoria

Nos enfocaremos a continuación en las técnicas de administración de memoria utilizadas en los primeros sistemas operativos, los cuales gestionan la memoria principal como un bloque continuo.

Este espacio disponible se dividía estratégicamente en bloques o particiones para permitir la ejecución de múltiples programas, optimizando su uso y asegurando un funcionamiento eficiente del sistema.

Si bien estos métodos son simples de gestionar también presentan algunas desventajas. Existen 2 tipos principales de gestión de particiones:

- Particiones Fijas
- Particiones Dinámicas o Variables

3.1.1 Particiones Fijas

En la mayoría de los esquemas para gestión de la memoria, se puede asumir que el sistema operativo ocupa alguna porción fija de la memoria principal y que el resto de la memoria principal está disponible para múltiples procesos.

El esquema más simple para gestionar la memoria disponible es repartirla en regiones con límites fijos.

Si bien el uso de particionamiento fijo es casi desconocido hoy en día, un ejemplo de un sistema operativo exitoso que sí utilizó esta técnica fue un sistema operativo de los primeros mainframes de IBM, el sistema operativo OS/MFT (Multiprogramming with a Fixed Number of Tasks; Multiprogramado con un número fijo de tareas).

Aquí las variantes pueden ser que todas las particiones tengan el **mismo tamaño** o que tengan **diferentes tamaños**.

3.1.1.1 Particiones con mismo tamaño

En este caso, todas las particiones tienen el mismo tamaño, por lo tanto cualquier programa con tamaño menor o igual que el tamaño de la partición puede cargarse en cualquier partición disponible.

Si no hay particiones disponibles no se podrá cargar a menos que se realice alguna operación de intercambio o swap o memoria virtual para liberar espacio.

Existen 2 dificultades en este sistema,

a. Un programa podría ser demasiado grande para caber en una partición. En este caso, el programador debe diseñar el programa con el uso de *overlays*, de forma que sólo se necesite una porción del programa en memoria principal en un momento determinado. Cuando se necesita un módulo que no está presente, el programa de usuario debe cargar dicho módulo en la partición del programa, superponiéndolo (overlaying) a cualquier programa o datos que haya allí.

b. La utilización de la memoria principal es ineficiente. Cualquier programa, sin importar lo pequeño que sea, ocupará una partición entera, generando un desaprovechamiento de la memoria. Esto se denomina **fragmentación interna.**

Algoritmos de reubicación

Con particiones del mismo tamaño, la ubicación de los procesos en memoria es trivial. En cuanto haya una partición disponible, un proceso se carga en dicha partición.

Debido a que todas las particiones son del mismo tamaño, no importa qué partición se utiliza. Si todas las particiones se encuentran ocupadas por procesos que no están listos para ejecutar, entonces uno de dichos procesos debe llevarse a disco para dejar espacio para un nuevo proceso (swap).

3.1.1.2 Particiones con diferentes tamaños

Los problemas que tiene la técnica de partición de tamaño fijo se pueden mejorar, aunque no resolver, utilizando particiones de tamaños diferentes. En el ejemplo de la figura 1, se puede ver que programas de 16 MB podrían entrar sin necesidad de overlays y las particiones menores de 8 MB permiten acomodar programas más pequeños minimizando la *fragmentación interna*.

Algoritmos de reubicación

Con particiones de diferente tamaño, hay dos formas posibles de asignar los procesos a las particiones. La forma más sencilla consiste en asignar cada proceso a la partición más pequeña dentro de la cual cabe.

En este caso, se necesita una cola para cada tamaño de partición, que mantenga procesos en disco destinados a dicha partición

La ventaja de esta técnica es que los procesos siempre se asignan de tal forma que se minimiza la memoria malgastada dentro de una partición (fragmentación interna).

Aunque esta técnica parece óptima desde el punto de vista de una partición individual, no es óptima desde el punto de vista del sistema completo.

En la Figura 1 , por ejemplo, se considera un caso en el que no haya procesos con un tamaño entre 12 y 16 MB en un determinado instante de tiempo. En este caso, la partición de 16 MB quedará sin utilizarse, incluso aunque se puede asignar dicha partición a algunos procesos más pequeños.

Por tanto, una técnica óptima sería emplear una única cola para todos los procesos. En el momento de cargar un proceso en la memoria principal, se selecciona la partición más pequeña disponible que puede albergar dicho proceso. Si todas las particiones están ocupadas, se debe llevar a cabo una decisión para enviar a swap a algún proceso. Tiene preferencia a la hora de ser expulsado a disco el proceso que ocupe la partición más pequeña que pueda albergar al proceso entrante. Es también posible considerar otros factores, como la prioridad o una preferencia por expulsar a disco procesos bloqueados frente a procesos listos.

El uso de particiones de distinto tamaño proporciona un grado de flexibilidad frente a las particiones fijas. Adicionalmente, se puede decir que los esquemas de particiones fijas son relativamente sencillos y requieren un soporte mínimo por parte del sistema operativo y una sobrecarga de procesamiento mínimo. Sin embargo, tiene una serie de desventajas:

- El número de particiones especificadas en tiempo de generación del sistema limita el número de procesos activos (no suspendidos) del sistema.

- Debido a que los tamaños de las particiones son preestablecidos en tiempo de generación del sistema, los trabajos pequeños no utilizan el espacio de las particiones eficientemente. En un entorno donde el requisito de almacenamiento principal de todos los trabajos se conoce de antemano, esta técnica puede ser razonable, pero en la mayoría de los casos resulta ineficiente.

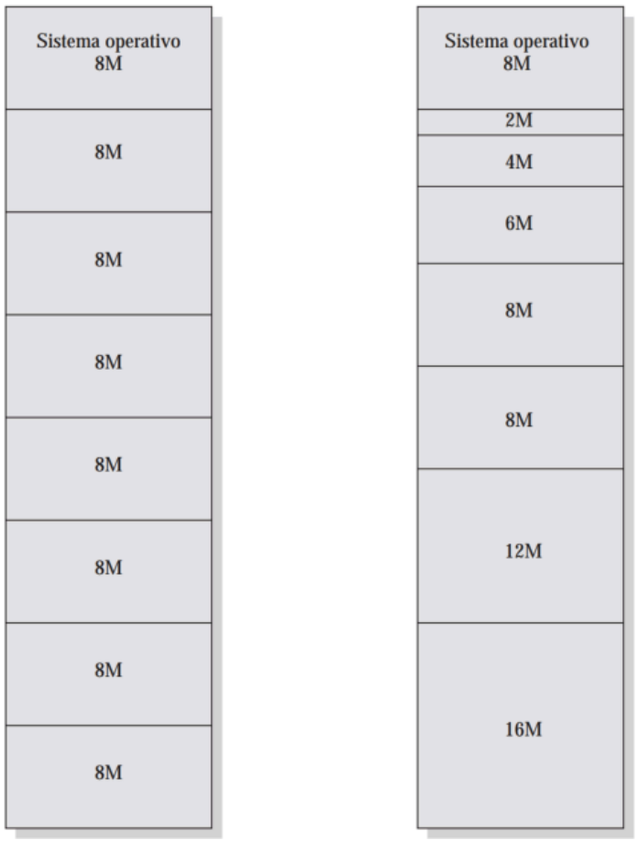


Figura 1. Ejemplos de particionamiento fijo de una memoria de 64 MB, una con mismo tamaño y otra con tamaños diferentes.[4]

3.1.2 Particiones variables o dinámicas

Con las particiones dinámicas, la memoria no está dividida en tamaños fijos de antemano. En cambio, cuando un programa necesita cargarse en la memoria, el sistema operativo **le asigna exactamente la cantidad de espacio que necesita**.

3.1.2.1 Algoritmos de reubicación en particiones dinámicas

Debido a que la compactación de memoria consume una gran cantidad de tiempo, el diseñador del sistema operativo debe ser inteligente a la hora de decidir cómo asignar la memoria a los procesos (cómo eliminar los huecos). A la hora de cargar o intercambiar un proceso a la memoria principal, y siempre que haya más de un bloque de memoria libre de suficiente tamaño, el sistema operativo debe decidir qué bloque libre asignar.

Tres algoritmos de colocación que pueden considerarse son **mejor-ajuste (best-fit)**, **primer-ajuste(first-fit) y siguiente-ajuste (next-fit)**. Todos, por supuesto, están limitados a escoger entre los bloques libres de la memoria

[4] Stallings, William (2005), Sistemas operativos. Aspectos internos y principios de diseño, Quinta edición.

principal que son iguales o más grandes que el proceso que se va a llevar a la memoria.[5]

Mejor-ajuste escoge el bloque más cercano en tamaño a la petición,dejando sobrantes de memoria mínimos. Una alternativa a este método es el llamado **Peor-ajuste (worse-fit),** que escoge el bloque más grande posible, generando una partición lo suficientemente grande para que pueda ser reutilizada por otro proceso.

Primer-ajuste comienza a analizar la memoria desde el principio y escoge el primer bloque disponible que sea suficientemente grande.

Siguiente-ajuste comienza a analizar la memoria desde la última colocación y elige el siguiente bloque disponible que sea suficientemente grande.

La *parte a* del esquema a continuación (figura 2), muestra un ejemplo de configuración de memoria después de un número de colocaciones e intercambios a disco. El último bloque que se utilizó fue un bloque de 22 MB, del cual se crea una partición de 14 MB. La *parte b* del mismo esquema, muestra la diferencia entre los algoritmos de mejor-, primer- y siguiente- ajuste a la hora de satisfacer una petición de asignación de 16 MB. *Mejor-ajuste* busca la lista completa de bloques disponibles y hace uso del bloque de 18 MB, dejando un fragmento de 2 MB. *Primer-ajuste* provoca un fragmento de 6 MB, y *siguiente-ajuste* provoca un fragmento de 20 MB.

[5] Stallings, William (2005), Sistemas operativos. Aspectos internos y principios de diseño, Quinta edición.

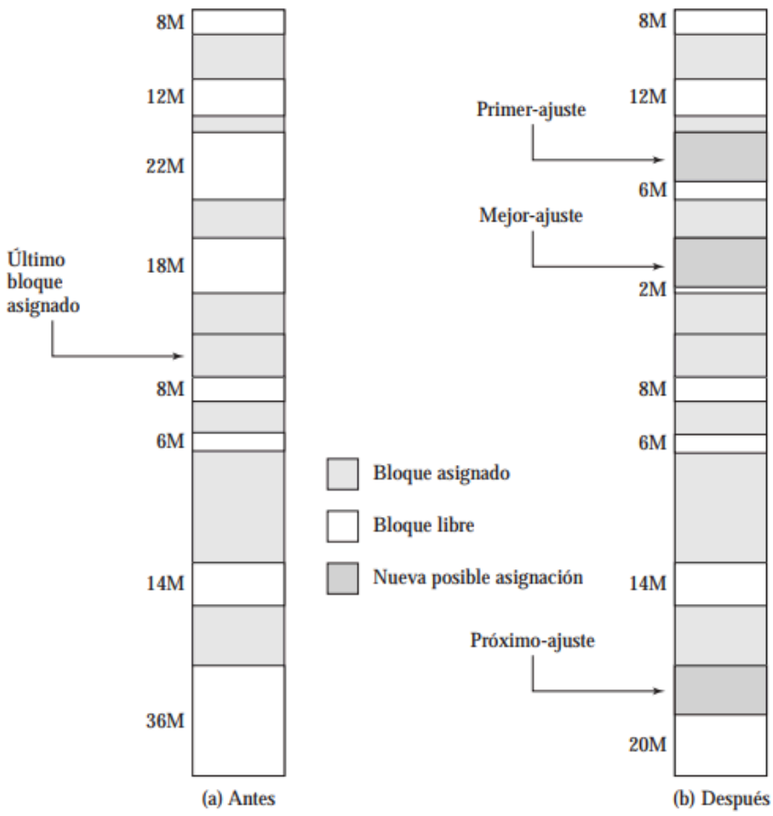


Figura 2. Ejemplo de configuración de la memoria antes y después de la asignación de un bloque de 16 MB.[6]

3.1.2.2 Fragmentación y Compactación

Antes de iniciar esta asignación, el sistema tenía 128 MB de memoria libre total, luego de la asignación, quedan 108 MB libres pero repartidos en bloques de diferentes tamaños. Dependiendo del algoritmo utilizado podrá variar el tamaño de la partición más grande.

En el caso de primer-ajuste y mejor-ajuste, la partición mayor será de 36 Bytes, mientras que el próximo-ajuste será de 22 MB.

A medida que pasa el tiempo la memoria se *fragmenta* cada vez más y puede no haber particiones lo suficientemente grandes para albergar al próximo trabajo.

Por ejemplo, si el sistema necesita ejecutar un proceso de 40 MB, si bien **hay memoria disponible suficiente** (108 MB), ésta no está situada en forma **contigua** y por lo tanto no podrá ejecutarse.

Este fenómeno se lo conoce como **fragmentación externa** y para solucionarlo requiere de un procedimiento llamado **compactación** que junta todos las particiones ocupadas y deja una única partición de memoria libre contigua listo

[6] Stallings, William (2005), Sistemas operativos. Aspectos internos y principios de diseño, Quinta edición.

para ser utilizada. Este es un proceso que consume muchos recursos del sistema y degrada la performance, por eso se trata de evitar esta situación en lo posible.

3.1.3 Cuadro Comparativo

Técnica	Descripción	Virtudes	Defectos
Particionamiento fijo	La memoria principal se divide en particiones estáticas en tiempo de generación del sistema. Un proceso se puede cargar en una partición con igual o superior tamaño.	Sencilla de implementar; poca sobrecarga para el sistema operativo.	Uso ineficiente de la memoria, debido a la fragmentación interna; debe fijarse el número máximo de procesos activos.
Particionamiento dinámico	Las particiones se crean de forma dinámica, de tal forma que cada proceso se carga en una partición del mismo tamaño que el proceso.	No existe fragmentación interna; uso más eficiente de memoria principal.	Uso ineficiente del procesador, debido a la necesidad de compactación para evitar la fragmentación externa.

Cuadro comparativo de particionamiento fijo y dinámico[7]

[7] Stallings, William (2005), Sistemas operativos. Aspectos internos y principios de diseño, Quinta edición.